# Configuration Schema Unification — Migration Guide

## Summary

All tool definitions now live under a **single `tools:` top-level key** in `config/tools.yaml`. The previous `web_tools:` namespace has been removed. `ConfigLoader.get_tool_config()` no longer searches multiple namespaces.

---

## BEFORE / AFTER — `config/tools.yaml`

### BEFORE (two top-level namespaces)

```yaml
# Network / AD / Surface / API tools lived here
tools:
  nmap:
    binary: nmap
    default_timeout: 600
  ffuf_api:
    binary: ffuf
    default_timeout: 300
  # ... 20+ tools ...

# Web tools had their own namespace
web_tools:
  whatweb:
    binary: whatweb
    default_timeout: 60
  ffuf:
    binary: ffuf
    default_timeout: 300
  nikto:
    binary: nikto
    default_timeout: 600
  # ... 9 tools ...
```

### AFTER (single unified namespace)

```yaml
tools:
  # ── Network Module Tools ──────────────────────────
  nmap:
    binary: nmap
    default_timeout: 600

  # ── AD Module Tools ───────────────────────────────
  enum4linux_ng:
    binary: enum4linux-ng
    default_timeout: 300

  # ── Surface Module Tools ──────────────────────────
  nmap_surface:
    binary: nmap
    default_timeout: 600

  # ── API Module Tools ──────────────────────────────
  ffuf_api:
    binary: ffuf
    default_timeout: 300

  # ── Web Module Tools ──────────────────────────────
  whatweb:
    binary: whatweb
    default_timeout: 60
  ffuf:
    binary: ffuf
    default_timeout: 300
  nikto:
    binary: nikto
    default_timeout: 600
  # ...all 30 tools under one key
```

**Key change:** The `web_tools:` top-level key is gone. All 30 tools sit directly under `tools:`, organized by module with comment headers.

---

## BEFORE / AFTER — `core/config_loader.py`

### BEFORE (conditional multi-namespace search)

```python
def get_tool_config(self, tool_name: str) -> dict:
    """Searches both tools.* and web_tools.* top-level namespaces."""
    data = self.load("tools")
    # Primary namespace
    result = data.get("tools", {}).get(tool_name)
    if result is not None:
        return result
    # Web-tools namespace (tools.yaml uses a separate top-level key)
    result = data.get("web_tools", {}).get(tool_name)
    if result is not None:
        return result
    return {}
```

### AFTER (single-path lookup — no fallbacks)

```python
def get_tool_config(self, tool_name: str) -> dict:
    """All tools live under the single 'tools' top-level key.
    No fallback namespaces — config is the source of truth."""
    data = self.load("tools")
    return data.get("tools", {}).get(tool_name, {})
```

**Key change:** Removed `web_tools` fallback branch. One namespace, one lookup, zero conditional logic.

---

## BEFORE / AFTER — Test Fixtures ( `tests/conftest.py` )

### BEFORE

```python
(cfg / "tools.yaml").write_text(
    "tools:\n"
    "  nmap:\n"
    "    binary: nmap\n"
    "    default_timeout: 600\n"
    "web_tools:\n"
    "  ffuf:\n"
    "    binary: ffuf\n"
    "    default_timeout: 300\n"
)
```

### AFTER

```python
(cfg / "tools.yaml").write_text(
    "tools:\n"
    "  nmap:\n"
    "    binary: nmap\n"
    "    default_timeout: 600\n"
    "  ffuf:\n"
    "    binary: ffuf\n"
    "    default_timeout: 300\n"
)
```

---

## BEFORE / AFTER — `tests/core/test_config_loader.py`

### BEFORE

```python
def test_load_tools(config_dir):
    data = loader.load("tools")
    assert "tools" in data
    assert "web_tools" in data     # <-- expected two namespaces
```

### AFTER

```python
def test_load_tools(config_dir):
    data = loader.load("tools")
    assert "tools" in data
    assert "web_tools" not in data  # <-- unified: no separate namespace
```

---

## Module Code Impact

No module code changes were required. All modules already used `self.config.get_tool_config("tool_name")`, which transparently resolves to the unified `tools:` namespace. The following modules were verified:

| Module | File | Calls `get_tool_config`? | Status |
|---|---|---|---|
| Web | `modules/web/base.py` | ✅ Yes (line 192) | Works — tool now found under `tools:` |
| API | `modules/api/base.py` | ✅ Yes (line 192) | Works — tool now found under `tools:` |
| AD | `modules/ad/base.py` | Uses config via phases | Works — no direct `get_tool_config` |
| Network | `modules/network/base.py` | Uses config via phases | Works — no direct `get_tool_config` |
| Surface | `modules/surface/base.py` | Uses config via phases | Works — no direct `get_tool_config` |

---

## Breaking Changes

| Change | Impact | Migration |
|---|---|---|
| `web_tools:` key removed from `tools.yaml` | Any code doing `data["web_tools"]` will get `KeyError` | Use `data["tools"]["tool_name"]` or `config.get_tool_config("tool_name")` |
| `ConfigLoader.get_tool_config()` no longer searches `web_tools` | If custom `tools.yaml` still uses `web_tools:`, those tools won't be found | Move all entries under `tools:` |

---

## Unified Tool Schema Reference

Every tool entry follows this consistent structure:

```yaml
tools:
  <tool_key>:
    binary: <executable_name>              # required
    description: "<what the tool does>"    # required
    required: true|false                   # required
    default_timeout: <seconds>             # required
    install_cmd: "<install command>"        # optional
    alt_binary: <alternative_name>         # optional
    detection: low|medium|high|very_high   # optional (tool-level default)
    opt_in_only: true                      # optional
    warning: "<safety note>"               # optional
    scan_profiles:                         # optional — for tools with scan modes
      <profile_name>:
        args: "<cli arguments>"
        timeout: <seconds>
        detection: low|medium|high
    modes:                                 # optional — for tools with operation modes
      <mode_name>:
        args: "<cli arguments>"
        timeout: <seconds>
        detection: low|medium|high
    safety:                                # optional — for dangerous tools
      max_tasks: <int>
      wait_time: <int>
```

All 30 tools now follow this schema consistently.